

XenChat

Privacy-First, Stealth Messaging Application

Project Report

Submitted for B.Sc. Information Technology

Patkar-Varde College, University of Mumbai

Submitted by:

Saniya Farooqui

Aniket Dubey

July 2026

Abstract

In an age where personal devices are frequently shared, borrowed, or physically accessible to others — at home, in hostels, or among peer groups — the boundary between private conversation and public visibility has grown increasingly thin. Conventional messaging applications either expose conversations in plain sight or rely on visibly “locked” states that draw attention to the very information they are meant to protect. A padlock icon or a “Private Chat” banner, ironically, signals to an onlooker that something worth hiding exists, defeating the purpose of privacy through the very act of announcing it.

XenChat is conceived to resolve this contradiction. The project's central idea is that true privacy in everyday messaging should be indistinguishable from ordinary, unremarkable activity — a conversation that looks like nothing more than odd or broken text to a passerby, rather than a conversation that is obviously guarded. Instead of hiding messages behind a lock that itself becomes a target of curiosity, XenChat disguises sensitive conversations in plain sight, revealing real content only through a deliberate, user-initiated action, and silently reverting to its disguised state once the user's attention moves elsewhere.

The motivation behind XenChat stems from real, everyday scenarios: a student texting on a shared family computer, roommates with limited physical privacy, or individuals who simply prefer that the content of their conversations remain their own business even if someone glances at their screen. Existing solutions in this space tend to fall into one of two categories — either heavyweight enterprise-grade encrypted messengers that are visibly built for security-conscious users, or casual chat apps with no meaningful privacy safeguards at all. Neither category addresses the everyday, casual need for a conversation to simply not look private while still being protected end-to-end.

The project further recognizes that privacy tools are often abandoned by casual users because they feel burdensome, clinical, or paranoia-inducing. XenChat addresses this by reframing privacy as part of a playful, game-like experience rather than a security chore, making the habit of protecting one's conversations feel natural and even enjoyable rather than a constant, anxiety-inducing routine.

In essence, XenChat sets out to solve three interconnected problems: the problem of visible privacy indicators drawing unwanted attention, the problem of privacy tools being too cumbersome for everyday casual use, and the problem of genuine duress situations where a user may be forced to unlock their device against their will. By addressing these problems together, the project aims to make private communication feel effortless, invisible, and safe for ordinary users rather than only for the technically inclined.

Chapter 1: Introduction

1.1 Theoretical Background

The idea for XenChat originates from an observation of how people behave around privacy on shared or semi-public devices. Most existing messaging platforms operate on an all-or-nothing model of privacy: either a conversation is fully visible to anyone who picks up the phone, or it is placed behind an explicit lock screen, PIN prompt, or a labelled “private” folder. Both extremes carry real drawbacks. A conversation left fully visible at risk of accidental or intentional exposure to family members, roommates, colleagues, or partners. A conversation locked behind a clearly labelled security feature, on the other hand, draws attention to itself — the very presence of a lock icon or a “Private Chat Enabled” banner tells an observer that something is being deliberately hidden, which can invite suspicion, questioning, or coercion to unlock it.

This gap between fully open and obviously locked communication is the theoretical foundation on which XenChat is built. The project explores a third state: conversations that are technically protected but visually camouflaged, so that a locked message looks like nothing more than a typo-ridden or oddly worded text rather than a security feature. This approach draws conceptually from the fields of steganography (hiding the existence of information rather than just its content) and applied cryptography (using audited, industry-standard encryption protocols such as the Signal Protocol to guarantee that message content is mathematically protected regardless of its visual disguise).

Working on this project was chosen because it sits at the intersection of several areas of genuine interest and relevance: practical cybersecurity and encryption, mobile application development, and interactive/game-based user experience design. It also responds to a lived, relatable problem rather than a hypothetical one — shared devices, peer group dynamics, and the discomfort of visible privacy indicators are everyday realities for students and young adults, making the project both technically substantial and personally meaningful to work on.

1.2 Objectives of the Project

The project is guided by the following objectives:

- To design and develop a mobile messaging application that protects sensitive conversations without visual signaling that protection is in place.
- To implement a reveal-and-reconceal mechanism where real message content is shown only through a deliberate user action and automatically hidden again after a short period of inactivity.
- To provide a genuine end-to-end encrypted communication channel so that message content is never readable by anyone other than the intended participants, including the server itself.
- To build a duress-response mechanism (Panic Mode) that allows a user to present a convincing decoy version of their chat history if forced to unlock their device under pressure.
- To make the habit of using privacy features enjoyable rather than burdensome, by weaving the underlying privacy mechanics into a set of playable in-app games.

- To support ephemeral and self-erasing communication, including timed self-destructing messages and temporary anonymous group rooms, for conversations that should leave no lasting trace.
- To build the application as a single cross-platform codebase capable of running on both Android and iOS devices.

The key features arising from these objectives include disguised message display (cipher blocks and AI-driven text scrambling), triple-tap reveal with silent auto-re-encryption, a fingerprint-and-PIN-protected Secret Vault, a duress-PIN-triggered Panic Mode with a physically separate decoy data partition, self-destructing messages, temporary anonymous rooms, and a Games Hub that turns privacy literacy into everyday play.

1.3 Purpose

XenChat is intended primarily for individuals who share devices or physical space with others but still want ordinary, everyday conversations to remain their own — students living in hostels or with family, young couples communicating on shared or borrowed phones, friend groups with sensitive group discussions, and privacy-conscious individuals in general who are uncomfortable with the idea of a visibly “locked” chat drawing attention. Unlike enterprise or activist-oriented secure messengers, XenChat is aimed at a mainstream, casual audience for whom privacy has so far felt like too much effort or too suspicious-looking to bother with.

The broader aim in bringing this project to market is to popularise the idea that privacy in messaging does not need to look like privacy at all — that protection and inconspicuousness can coexist in a single, approachable, even entertaining application. By packaging serious cryptographic protection inside a playful, game-themed interface, the project aims to appeal to a demographic that would otherwise never adopt a dedicated “secure messenger.”

1.4 Scope, Limitations & Availability

Scope and Availability

XenChat is designed as a cross-platform mobile application, available on both Android and iOS through a single React Native codebase. It is useful in any context where a user's device may be seen, borrowed, or physically inspected by someone else, including but not limited to: shared family or hostel devices, workplaces where phones may be glanced at by colleagues, and situations where a user is under social or coercive pressure to unlock their phone. It is equally useful for casual users who simply prefer a playful, gamified messaging experience over a conventional chat interface.

Limitations and Disadvantages

- Screenshot detection is only reliably achievable on Android; iOS platform restrictions limit this to a best-effort, non-guaranteed feature.
- The 4-character passcode fallback, while attempt-limited, is inherently weaker than biometric authentication and remains a potential point of compromise if biometrics are unavailable.

- The AI-driven Ghost Language transformation depends on a third-party LLM API at message-send time; unavailability of this service causes a fallback to the less immersive cipher-block display.
- A forgotten passcode with no biometric enrolled results in permanent loss of Privacy Mode chat history, since no server-side key escrow is maintained by design.
- The Panic Mode decoy mechanism depends on strict separation of data partitions at the storage layer; any implementation weakness here could compromise its forensic credibility.
- As a subscription-only, ad-free model, monetization depends entirely on conversion to paid tiers, which may limit short-term revenue during early adoption.

1.5 Expected Achievements

By the end of development, XenChat is expected to deliver a fully functional, cross-platform messaging application in which sensitive conversations are cryptographically protected end-to-end while remaining visually indistinguishable from ordinary, unremarkable text. The end goal is a product that a user can hand over to another person — even under duress — without genuine private content ever being exposed, while the everyday experience of using the app feels closer to a game than a security tool. Ultimately, the project aims to demonstrate that meaningful, everyday privacy can be made invisible, effortless, and even enjoyable, rather than remaining the preserve of technically sophisticated or visibly “locked-down” applications.

Chapter 2: Technologies & Languages

2.1 Chosen Project Domain

XenChat falls within the domain of Mobile Application Development, specifically cross-platform real-time messaging systems, combined with applied cryptography (end-to-end encryption) and artificial intelligence (LLM-based text transformation). It also draws on real-time systems engineering for its WebSocket-based chat and multiplayer game infrastructure.

2.2 Comparative Analysis

Frontend Framework Comparison

Criteria	React Native (Expo)	Flutter	Native (Kotlin/Swift)
Codebase	Single, JS/TS	Single, Dart	Two separate codebases
Team familiarity	High (existing JS skill)	None (new language)	Low
Crypto library support	Strong (libsignal bindings)	Weaker native bindings	Strongest, but costly
Animation capability	Reanimated + Skia, sufficient	Good, but less proven for this use	Best, but high effort
Development speed	Fast (Expo tooling)	Fast	Slow (2x work)
Verdict	Selected	Rejected	Rejected

Backend Runtime Comparison

Criteria	Node.js (Express)	Django (Python)	Spring Boot (Java)
Language consistency with frontend	Yes (JS/TS both ends)	No	No
Real-time / WebSocket support	Excellent (Socket.io)	Weaker out-of-the-box	Good, but heavier setup
Development speed for MVP	Fast	Moderate	Slow (boilerplate-heavy)
Community & ecosystem	Very large	Large	Large (enterprise-focused)
Verdict	Selected	Rejected	Rejected

Database Comparison

Criteria	PostgreSQL	MongoDB	Firebase Firestore
----------	------------	---------	--------------------

Data model fit	Relational, suits structured user/game data	Flexible schema, less needed here	Flexible, real-time listeners
ACID compliance	Yes	Partial	Partial
Vendor lock-in	None	Low	High
Cost at scale	Predictable	Predictable	Can grow expensive
Verdict	Selected	Rejected	Rejected

Encryption Library Comparison

Criteria	libsignal-protocol-js	Custom crypto implementation	Third-party paid E2EE SaaS
Security auditing	Independently audited (Signal Protocol)	Unaudited, high risk	Varies by vendor
Cost	Free, open-source	Free but risky	Often paid/licensed
Control over implementation	Full	Full	Limited (vendor lock-in)
Industry trust	Very high (used by Signal, WhatsApp)	None	Moderate
Verdict	Selected	Rejected	Rejected

2.3 Technologies & Languages Used

The following technologies form the core stack of the project, spanning the frontend mobile client, backend services, real-time communication layer, encryption module, and supporting infrastructure.

Frontend

- React Native (Expo) — cross-platform mobile framework for building the Android and iOS client from a single JavaScript/TypeScript codebase.
- React Navigation v7 — bottom tab and stack navigation across the app's five main sections.
- Zustand — lightweight global state management.
- React Query — server-state fetching, caching, and synchronization.
- Reanimated 3 and Skia — high-performance animations used for the glitch/decode reveal effect.
- MMKV — fast, encrypted local on-device storage.

Backend

- Node.js with Express — REST and GraphQL hybrid API server.

- Socket.io — real-time WebSocket layer for messaging, presence, typing indicators, and real-time game state.
- Google OAuth 2.0 and Firebase Auth (phone OTP) — authentication.

Database & Storage

- PostgreSQL — relational database for user accounts, contacts, game scores, and leaderboards.
- Redis — session tokens, Socket.io room state, and Anonymous Room TTL/lifecycle management.
- Cloudflare R2 (S3-compatible object storage) — encrypted media storage.

Encryption / Security

- libsodium-protocol-javascript — implementation of the audited Signal Protocol (X3DH key exchange and Double Ratchet) for end-to-end encryption.
- react-native-libsodium — on-device cryptographic key management.
- react-native-biometrics — unified fingerprint and Face ID authentication API.

AI Layer

- Server-side LLM API integration — powers the Pro-tier AI Ghost Language feature, rewriting message text in the sender's own tone while scrambling its literal meaning.

Payments

- Razorpay — UPI, card, and net-banking subscription billing for Indian users.
- Stripe — subscription billing for international users.

Development Tools & IDE

- Visual Studio Code — primary IDE for both frontend and backend development.
- Git and GitHub — version control and collaborative development across the two-person team.
- Expo Application Services (EAS) — build and release pipeline for Android and iOS binaries.
- Thunder Client — API endpoint testing during backend development.

2.4 Justification of Chosen Technologies

React Native (Expo) was selected over Flutter and native Android/iOS development because it allows a single JavaScript/TypeScript codebase to target both major mobile platforms simultaneously, which is essential for a small two-person student team working under a fixed deadline. Expo further simplifies build distribution and over-the-air updates, and its ecosystem offers mature support for the Reanimated and Skia libraries needed for the app's signature glitch-decode animations — a capability that would have required significantly more native crypto-binding work under Flutter.

Node.js with Express was chosen over Django and Spring Boot primarily because it allows the same language, JavaScript/TypeScript, to be used across both the frontend and backend, reducing context-switching for a small team. Its native event-driven architecture also integrates naturally with Socket.io,

which is central to XenChat's real-time chat and multiplayer game requirements — a capability that is comparatively harder to achieve with Django's synchronous-first design.

PostgreSQL was selected over MongoDB and Firebase Firestore because XenChat's data — user accounts, contacts, game scores, and leaderboards — is inherently relational, and PostgreSQL's ACID compliance provides stronger guarantees for this structured data than a flexible-schema NoSQL store would, while avoiding the vendor lock-in associated with Firebase.

Socket.io was chosen over raw WebSockets or Firebase Realtime Database for its automatic reconnection handling, built-in room abstraction (used directly for Anonymous Rooms and the Neon Duel game), and proven compatibility with React Native.

libsignal-protocol-javascript was selected over building a custom encryption scheme because it implements the Signal Protocol, an independently audited and widely trusted standard for end-to-end encrypted messaging. Writing custom cryptography for a messaging application is a well-recognised anti-pattern and security liability, so relying on a vetted library was considered non-negotiable.

An LLM API was chosen for the AI Ghost Language feature over static word-shuffling algorithms because a language model can preserve the sender's natural tone and vocabulary while scrambling the literal meaning, producing a far less detectable disguise than a mechanically shuffled sentence would.

Razorpay and Stripe were jointly selected to cover, respectively, India-first payment methods (UPI, net banking) and international card-based billing, ensuring the subscription model is accessible to both the primary domestic user base and any future international users.

2.5 References

The following sources were consulted, or are planned to be consulted, during the research, design, and development of this project:

- Signal Protocol Documentation — <https://signal.org/docs/>
- React Native Official Documentation — <https://reactnative.dev/docs/getting-started>
- Expo Documentation — <https://docs.expo.dev/>
- Socket.io Documentation — <https://socket.io/docs/v4/>
- Node.js Official Documentation — <https://nodejs.org/en/docs/>
- PostgreSQL Official Documentation — <https://www.postgresql.org/docs/>
- Redis Documentation — <https://redis.io/docs/latest/>
- libsignal-protocol-javascript (GitHub) — <https://github.com/signalapp/libsignal-protocol-javascript>
- Razorpay Developer Documentation — <https://razorpay.com/docs/>
- Stripe Documentation — <https://stripe.com/docs>
- Cloudflare R2 Documentation — <https://developers.cloudflare.com/r2/>

- Anthropic Claude API Documentation — <https://docs.claude.com>
- OWASP Mobile Application Security Guidelines — <https://owasp.org/www-project-mobile-app-security/>